

CSE401: Numerical Analysis, Simulation and Modeling

Introduction to Simulation and Modeling, Discrete Event Simulation Models;
Steps in a Simulation Study; Model Validation and Verification

Nafis Tahmid

Department of CSE, BUET

You have a system.

You want to know how it behaves.

You can't (or shouldn't) experiment with the real thing.

What do you do?

You simulate.

Why Not Just Experiment with the Real System?

Imagine you manage a hospital emergency room. You want to know:
“What happens if I reduce the number of doctors from 5 to 3?”

Why Not Just Experiment with the Real System?

You *could* just fire two doctors and see what happens.

Possible outcomes:

- Patients wait 4 hours instead of 30 minutes
- People get really, really angry
- The hospital gets sued
- You get fired

Why Not Just Experiment with the Real System?

Or... you could build a **model** of the ER,
run it on a computer, and find out *before* making anyone suffer.

Key Insight

Simulation lets you make mistakes cheaply and safely.
The computer doesn't file lawsuits.

Another Example: The Port of Western Australia

- A port was being redesigned.
- Each loading/unloading berth costs roughly **US\$300 million**.
- You can't build one, discover it's inadequate, and say "Oops, let's try again."
- So they simulated it first.

Moral

Simulation: because some experiments are too expensive, too dangerous, or too embarrassing to run in real life.

So What Exactly IS Simulation?

Definition

A **simulation** is the **imitation** of the operation of a real-world process or system **over time**.

Let's break this down:

- **Imitation** — we create a *fake* version of something real
- **Real-world process or system** — something that exists (or could exist)
- **Over time** — we watch how it evolves, step by step

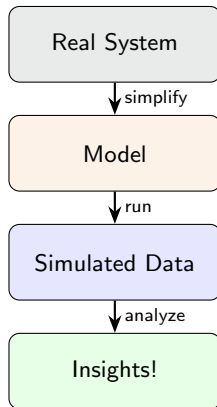
In Plain English

We generate an **artificial history** of a system, observe it, and draw conclusions about how the real system would behave.

The Artificial History Idea

Think of it like a **rehearsal** for reality:

- We create a model (a simplified version of the real thing)
- We “run” the model on a computer
- We collect data from the model as if we were observing the real system
- We use that data to estimate how the real system performs



Simulation vs. Analytical Solutions

Wait — can't we just use *math* to solve everything?

Sometimes, yes. If the system is simple enough, we can write equations and solve them directly. This is called an **analytical solution**.

Simulation vs. Analytical Solutions

Example of an analytical solution:

Distance = Rate $\times$ Time.

If you drive at 60 km/h for 2 hours, you go 120 km. Done.

No simulation needed. Math wins.

Simulation vs. Analytical Solutions

But now consider:

“What is the average waiting time for customers at a bank with 3 tellers, where customers arrive randomly and service times vary?”

The math gets... *complicated*.

Real-world systems have so many interacting parts that analytical solutions are often impossible.

Simulation vs. Analytical Solutions

When to use what

Simple system	Use math (analytical solution)
Complex system	Use simulation

Most real-world systems are complex.

That's why simulation is so widely used.

When IS Simulation the Right Tool?

1. The system is too **complex** to solve mathematically
2. You want to **experiment** without disrupting the real system
3. The system **doesn't exist yet** (still in design phase)
4. You want to study the system under **extreme conditions** ("What if 1000 patients arrive at once?")
5. You want to **compress or expand** time (simulate a year in minutes, or slow down a nanosecond process)
6. You want to **train people** without real-world risk (flight simulators, military exercises)
7. You want to **test new designs** before committing resources

When Simulation is NOT Appropriate

Rule 1: **Don't simulate if common sense suffices.**

Example: Customers arrive at a rate of 100/hour. Each server handles 12/hour.

Minimum servers needed? $\lceil 100/12 \rceil = 9$.

You don't need a simulation for that. You need a calculator.

When Simulation is NOT Appropriate

Rule 2: **Don't simulate if the problem can be solved analytically.**

If there's a clean formula, use it. It's faster and exact.

Simulation gives you *estimates*; math gives you *answers*.

When Simulation is NOT Appropriate

Rule 3: **Don't simulate if direct experiments are cheaper.**

A fast-food chain wanted to test a new ordering station.

Instead of simulating, they just... put a person with a handheld terminal outside.

Cost: almost nothing. Result: immediate.

Sometimes the simplest approach is the best.

When Simulation is NOT Appropriate

More reasons NOT to simulate:

- The cost of the study exceeds the potential savings
- There isn't enough time or budget
- No data is available (not even estimates)
- The system behavior is too complex to even *define*

Where is Simulation Used? (Everywhere.)

Manufacturing

- Assembly lines
- Wafer (silicon) fabrication
- Bottleneck analysis

Healthcare

- Emergency room design
- Operating room scheduling
- Disease spread modeling

Transportation

- Airport runway management
- Traffic flow
- Port operations

Military

- Weapons evaluation
- Training simulations
- Logistics planning

Business

- Call centers
- Supply chains
- Inventory management

And more. . .

- Computer networks
- Construction projects
- Even predator–prey ecosystems!

Systems, Models, and Classifications

What is a “System”?

Definition

A **system** is a collection of **entities** (people, machines, etc.) that **act and interact** together toward the accomplishment of some logical end.

Examples:

- A **bank**: tellers + customers + queues + rules
- A **factory**: machines + workers + parts + conveyors
- A **hospital**: doctors + nurses + patients + beds
- A **computer network**: routers + messages + protocols

Key point: What you consider “the system” depends on what you’re studying!

The System Boundary

Consider a bank.

If you want to study **waiting times for regular customers**:

→ The system = tellers + customers in line

The loan officer? The safe-deposit boxes? Not part of your system.

The System Boundary

But if you want to study **the entire bank operation**:

→ The system = tellers + loan officers + safe-deposit + ATMs + ...

The system boundary depends on the **purpose of the study**.

The System Boundary

System Environment

Everything outside the system boundary is the **system environment**.

The environment can *affect* the system, but the system doesn't control it.

Example: Customer arrival patterns are part of the bank's *environment* (the bank can't control when people walk in).

Components of a System

Every system has building blocks. Let's meet them:

Entity: An object of interest in the system

Attribute: A property of an entity

Activity: A time period of specified length

Event: An instantaneous occurrence that may change the system state

State variable: A variable describing the system at any time

Bank Example

Entity: Customers

Attribute: Account balance

Activity: Making a deposit

Event: Customer arrives; customer departs

State variable: Number of busy tellers; number waiting

More System Component Examples

System	Entity	Attribute	Activity	Event	State Var.
Bank	Customers	Balance	Deposit	Arrival	# busy tellers
Rail	Riders	Origin	Traveling	Arr. at stn.	# waiting
Factory	Machines	Speed	Stamping	Breakdown	Status
Network	Messages	Length	Transmit	Arr. at dest.	# waiting
Inventory	Warehouse	Capacity	Withdraw	Demand	Stock level

Notice

You can only list components once you know the **purpose of the study**. Different purposes → different components matter.

State Variables

The **state** of a system is the collection of variables that describe the system at any point in time.

Think of it as a *snapshot*: if someone gave you these numbers, you could reconstruct exactly what's happening.

State Variables

Bank example:

- Number of busy tellers: 2
- Number of customers in queue: 5
- Time of arrival of the next customer: 10:23 AM

With just these three numbers, you know quite a lot about what's happening at the bank right now.

State Variables

Important: State variables change only when **events** occur.

Between events? Nothing changes. The system just... sits there.

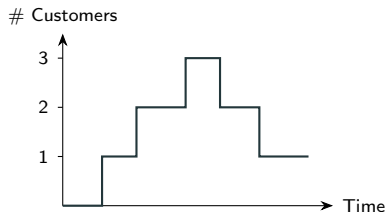
(Well, people are being served, but from the model's perspective, the *state numbers* don't change until someone arrives or departs.)

Discrete vs. Continuous Systems

Discrete System

State variables change at **discrete** (separated) points in time.

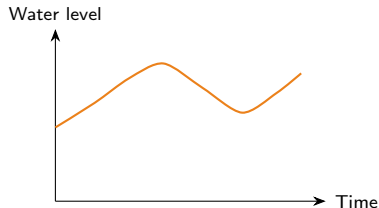
Example: A bank. The number of customers changes only when someone arrives or departs.



Continuous System

State variables change **continuously** over time.

Example: Water level behind a dam. It rises and falls smoothly.



Most real systems are a mix, but we classify by the **predominant** type of change.

What is a “Model”?

Definition

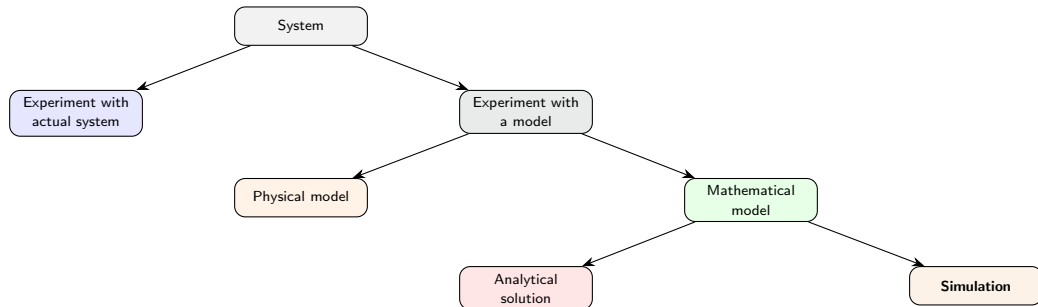
A **model** is a **representation** of a system for the purpose of studying that system.

A model is, by definition, a **simplification**. We don't model everything — just the parts that matter for our question.

Analogy

A map of Dhaka is a model of Dhaka. It doesn't show every brick, every person, every stray cat. But it shows roads, landmarks, and distances — which is what you need for navigation.

How to Study a System



Simulation is what we use when we have a mathematical model that's too complex to solve analytically.

Types of Simulation Models

Simulation models are classified along **three dimensions**:

1. **Static** vs. **Dynamic**

- Static: represents a system at a single point in time (e.g., Monte Carlo)
- Dynamic: represents how a system evolves over time (e.g., bank from 9 AM to 4 PM)

2. **Deterministic** vs. **Stochastic**

- Deterministic: no randomness; same inputs → same outputs
- Stochastic: contains random variables; outputs are *estimates*

3. **Discrete** vs. **Continuous**

- Discrete: state changes at separated points in time
- Continuous: state changes continuously

What We Focus On: Discrete-Event Simulation

Our Focus

- **Dynamic** — they evolve over time
- **Stochastic** — they involve randomness
- **Discrete** — state changes at discrete points

This combination is called **Discrete-Event Simulation** (DES).

Why stochastic?

Because the real world is random! Customers don't arrive at exact intervals. Service doesn't always take the same time. Machines break down unpredictably. Ignoring this randomness gives you a dangerously wrong model.

Deterministic vs. Stochastic: An Intuition

Deterministic: A dentist's office where every patient arrives exactly at their scheduled time.

If the schedule says 10:00, 10:30, 11:00 — that's exactly when they show up.

No randomness. Perfectly predictable.

(Also: completely unrealistic. When was the last time you arrived at the dentist on time?)

Deterministic vs. Stochastic: An Intuition

Stochastic: A bank where customers walk in randomly.

Sometimes 3 people arrive in one minute, then nobody for 5 minutes.

Service takes 2 minutes for one customer, 8 minutes for the next.

The randomness is the whole point — it creates queues, delays, and interesting behavior.

Deterministic vs. Stochastic: An Intuition

Key Consequence of Stochastic Models

Because inputs are random, **outputs are also random**.

Run the same model twice with different random numbers → different results.

So simulation outputs are **estimates**, not exact answers.

We need statistics to interpret them properly.

Discrete-Event Simulation

What is Discrete-Event Simulation?

Definition

Discrete-event simulation (DES) is the modeling of a system in which the state variables change only at a **discrete set of points in time**.

These points in time are called **events**. Between events, nothing changes.

Think of a Bank Again

- Customer arrives at 10:03 → state changes (queue grows by 1)
- Service completes at 10:07 → state changes (queue shrinks by 1)
- Between 10:03 and 10:07? The queue length stays the same.

Nothing interesting happens *between* events.

The Simulation Clock

- Every DES model has a variable called the **simulation clock**.
- It records the current value of simulated time.
- **Important:** Simulated time $\neq$ actual computer time!
- You could simulate 24 hours of bank operation in 0.01 seconds of computer time.
- Or simulate 1 nanosecond of a computer network in 10 minutes of computer time.

How Does the Clock Move Forward?

Two approaches:

1. **Next-Event Time Advance** (the one we use)
 - Jump the clock from one event to the next
 - Skip over “boring” periods where nothing happens
 - Used by virtually all modern simulation software
2. **Fixed-Increment Time Advance**
 - Advance the clock by a fixed amount Δt each step
 - Check if any events occurred during $[t, t + \Delta t)$
 - Wasteful: lots of steps where nothing happens

Next-Event Time Advance

The idea:

1. Start the clock at 0
2. Determine when future events will occur
3. Jump the clock to the *nearest* upcoming event
4. Process that event (update state, schedule new events)
5. Repeat until done

Next-Event Time Advance

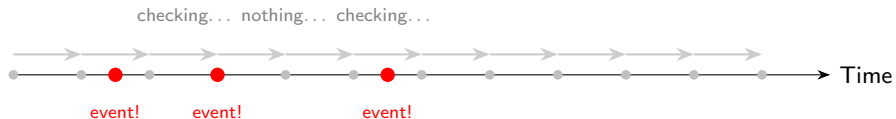
Visualizing it:



The clock **jumps** from event to event. Idle periods are skipped entirely. This is efficient!

Next-Event Time Advance

Contrast with fixed-increment:



Fixed-increment wastes time checking steps where nothing happens.

Components of a DES Model

Every DES model (when programmed) has these parts:

System state: Variables describing the system right now

Simulation clock: Current simulated time

Event list: List of upcoming events and when they'll occur

Statistical counters: Variables accumulating performance data

Initialization routine: Sets everything up at time 0

Timing routine: Finds the next event and advances the clock

Event routines: Update state when an event occurs

Library routines: Generate random numbers

Report generator: Computes and prints results

Main program: Orchestrates everything

The Event List

The **event list** (also called the *future event list*) is simply a list of events that are **scheduled** to happen in the future, along with the **time** each will occur.

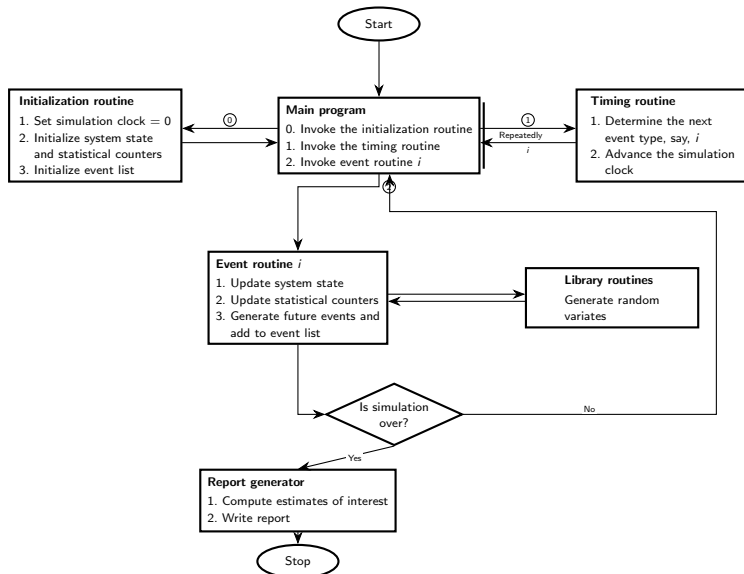
The Event List

Example:

Event Type	Scheduled Time
Customer 4 arrives	10:15
Customer 2 finishes service	10:08
Customer 5 arrives	10:22

The timing routine scans this list and picks the earliest event (Customer 2 finishes at 10:08).

Flow of Control in a DES



Flow of Control in a DES

1. **Start:** The main program invokes the **initialization routine**
 - Set simulation clock = 0
 - Initialize system state and statistical counters
 - Schedule initial events in the event list
2. **Repeatedly** (this is the main loop):
 - 2.1 Main program calls **timing routine** → determines next event type i and advances clock
 - 2.2 Main program calls **event routine** i → updates state, updates counters, schedules future events (may call **library routines** to generate random variates)
 - 2.3 Check: is the simulation over? → **No:** go back to step 1 of this loop → **Yes:** invoke the **report generator** and **stop**

Single-Server Queueing System

Introduction to Queueing

What's a queue?

A line. That's it. You stand in it. You wait. You get served. You leave.

You've been in hundreds of queues in your life.

Now we're going to *study* them.

Why Study Queues?

Queues form whenever demand exceeds supply, even temporarily.

Understanding them lets us design better systems: shorter waits, lower costs, happier people.

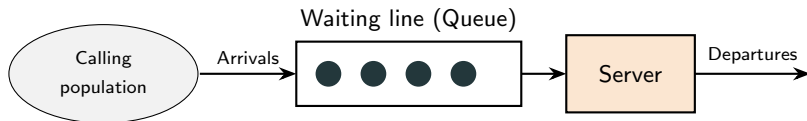
Queues Are Everywhere

- Customers waiting at a bank teller
- Cars waiting at a traffic signal
- Patients waiting at a hospital
- Jobs waiting for a CPU in a computer
- Phone calls waiting for an agent at a call center
- Packets waiting at a network router

The Common Thread

In every case: **someone/something arrives, needs a resource, and might have to wait** because the resource is busy.

Components of a Queueing System



A queueing system has these components:

1. **Calling population:** Where do the customers come from?
2. **Arrival process:** How do customers arrive? (Interarrival times)
3. **Service mechanism:** How many servers? How long does service take?
4. **Queue discipline:** Who gets served next? (FIFO, priority, etc.)
5. **System capacity:** Is there a limit on how many can wait?

Arrival Process: How Customers Show Up

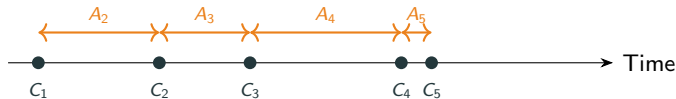
Customers arrive **one at a time** at **random** intervals.

The time between two consecutive arrivals is called the **interarrival time**.

We denote the interarrival time between customer $i-1$ and customer i as A_i .

Arrival Process: How Customers Show Up

Example:



Notice: interarrival times are **not equal**. Sometimes customers arrive in bursts (A_5 is tiny), sometimes there are long gaps (A_4 is large). This randomness is what makes queueing interesting.

Arrival Process: How Customers Show Up

Arrival time of customer i : the *cumulative* sum of interarrival times.

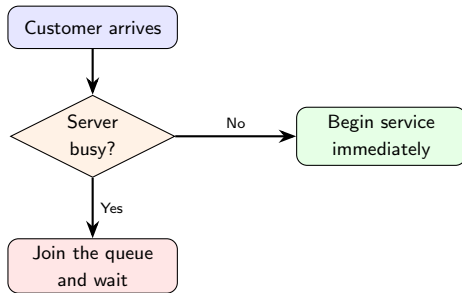
If the first customer arrives at time A_1 , then:

Arrival time of customer 1 = A_1

Arrival time of customer 2 = $A_1 + A_2$

Arrival time of customer $i = \sum_{j=1}^i A_j$

What Happens When a Customer Arrives?

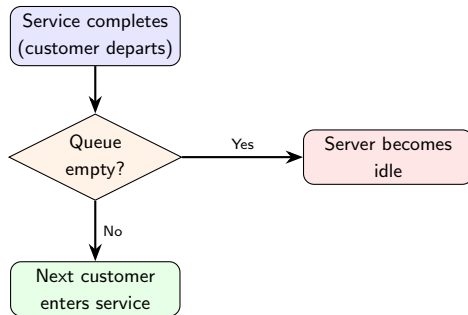


Two Possible Outcomes

Server idle: Lucky customer! Zero delay. Goes straight to service.

Server busy: Customer joins the end of the queue. Waits until it's their turn.

What Happens When Service Completes?



Two Possible Outcomes

Queue empty: Server becomes idle. We set next departure time to ∞ .

Queue not empty: First person in line moves to the server. Generate their service time and schedule their departure.

Our Setup: The Single-Server System

- **One server** (e.g., one barber, one ATM, one teller)
- Customers arrive **one at a time**, at **random** intervals
- If server is **idle** $\rightarrow$ customer goes straight to service (delay = 0)
- If server is **busy** $\rightarrow$ customer joins the end of the queue
- Queue discipline: **FIFO** (first in, first out)
- We start **empty and idle**: no customers, server idle

Notation

A_i interarrival time between customer $i-1$ and i

S_i service time of customer i

D_i delay in queue of customer i (waiting time *before* service)

Important: Delay $\neq$ Total Time in System

Let's be very precise about what **delay in queue** means.

Delay D_i = the time customer i spends *waiting in the queue*, **not** being served.

If you arrive and the server is idle, your delay is **zero** — you don't wait at all.

Important: Delay $\neq$ Total Time in System

Total time in system would be:

$$\text{Total time} = D_i + S_i \quad (\text{delay} + \text{service time})$$

We focus on D_i (delay) because it measures the *inefficiency* of the system. Service time S_i is inherent to the task — you can't make a haircut take zero minutes. But waiting time? That's waste.

What the Computer Tracks

To simulate this system, we need to maintain these variables:

1. **Clock**: Current simulated time
2. **Server status**: 0 (idle) or 1 (busy)
3. **Number in queue**: How many customers are waiting
4. **Times of arrival**: A list storing the arrival time of each customer in the queue (we need this to compute delays later!)
5. **Time of last event**: When the previous event occurred (needed for area calculations)
6. **Event list**: Times of next arrival and next departure

Statistical Counters

Number delayed (so far)	Total delay accumulated
Area under $Q(t)$ (so far)	Area under $B(t)$ (so far)

What Do We Want to Measure?

Three things:

1. How long do customers **wait**?
2. How **crowded** is the queue?
3. How **busy** is the server?

Let's define each one precisely, with the intuition behind the formula.

Measure 1: Average Delay in Queue

Average delay in queue, $\hat{d}(n)$:

After n customers have completed their delays, what was the average waiting time?

$$\hat{d}(n) = \frac{D_1 + D_2 + \cdots + D_n}{n} = \frac{\sum_{i=1}^n D_i}{n}$$

Measure 1: Average Delay in Queue

Example:

$$D_1 = 0, D_2 = 0.8, D_3 = 1.0, D_4 = 0, D_5 = 0.9, D_6 = 3.0$$

$$\hat{d}(6) = \frac{0 + 0.8 + 1.0 + 0 + 0.9 + 3.0}{6} = \frac{5.7}{6} = 0.95$$

On average, a customer waited about 0.95 time units before being served.

Measure 2: Time-Average Number in Queue

Time-average number in queue, $\hat{q}(n)$:

On average over time, how many customers are standing in the queue?

This is **not** a simple average! We need a **time-weighted average**.

Measure 2: Time-Average Number in Queue

Why not a simple average?

Suppose the queue has 0 customers for 9 hours and 10 customers for 1 hour.

Simple average of observed values: $\frac{0+10}{2} = 5$. **Wrong!**

The queue was empty 90% of the time! The correct answer should be closer to $\frac{0 \times 9 + 10 \times 1}{10} = 1.0$.

We must weight each queue length by **how long** it lasted.

Measure 2: Time-Average Number in Queue

Define $Q(t)$ = number of customers in the queue at time t (not counting the one in service).

$Q(t)$ is a **step function** — it changes only at event times, and stays constant between events.

The time-average is:

$$\hat{q}(n) = \frac{\int_0^{T(n)} Q(t) dt}{T(n)}$$

where $T(n)$ is the time the simulation ends.

Measure 2: Time-Average Number in Queue

Don't panic about the integral!

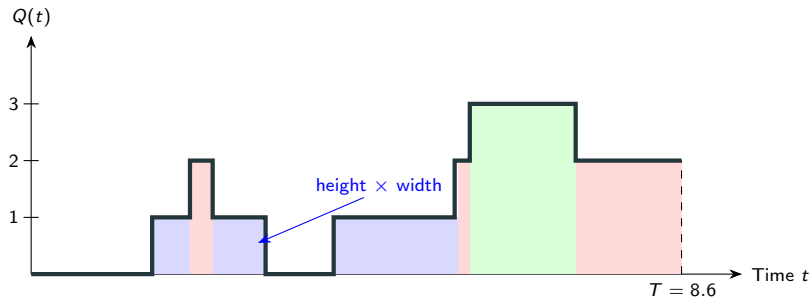
Since $Q(t)$ is a step function (constant between events), the “integral” is just a sum of **rectangles**:

$$\int_0^{T(n)} Q(t) dt = \sum_{\text{each interval}} (\text{queue length during interval}) \times (\text{duration of interval})$$

Each rectangle has:

- **Height** = value of $Q(t)$ during that interval
- **Width** = time from one event to the next

The $Q(t)$ Step Function



$$\text{Shaded area} = \int_0^T Q(t) dt = 9.9$$

$$\hat{q}(6) = \frac{9.9}{8.6} = 1.15$$

On average, about 1.15 customers were in the queue at any given moment.

Measure 3: Server Utilization

Server utilization, $\hat{u}(n)$: What fraction of time is the server busy?

Define the **busy function**:

$$B(t) = \begin{cases} 1 & \text{if server is busy at time } t \\ 0 & \text{if server is idle at time } t \end{cases}$$

Measure 3: Server Utilization

Then server utilization is just the time-average of $B(t)$:

$$\hat{u}(n) = \frac{\int_0^{T(n)} B(t) dt}{T(n)} = \frac{\text{total time server was busy}}{T(n)}$$

Since $B(t)$ is always 0 or 1, the integral is just the **total busy time**.

This gives a number between 0 and 1:

- $\hat{u} = 0$: server was idle the entire time (no customers ever came)
- $\hat{u} = 1$: server was busy the entire time (never rested)
- $\hat{u} = 0.9$: server was busy 90% of the time

Measure 3: Server Utilization

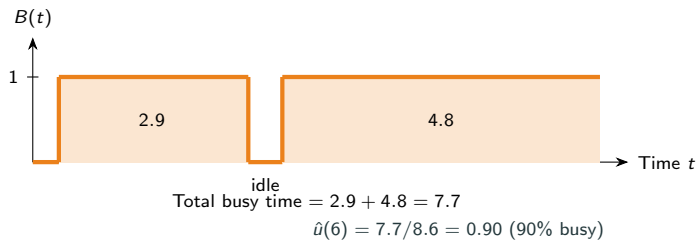
Why utilization matters:

- Near 100%: server is a **bottleneck** — queues will grow
- Very low: **excess capacity** — you're paying for an idle resource
- Sweet spot: depends on the system, but usually 70–85%

Rule of Thumb

If arrival rate $>$ service rate, the queue will grow without bound (“explosive” queue).

The $B(t)$ Function



How We Actually Compute the Areas

Key insight: We don't draw the graph and measure areas with a ruler!

We **accumulate** the areas as the simulation runs, event by event.

At each event, **before** updating the state, we add a rectangle:

$$\text{new area} = \underbrace{\text{current value of } Q(t) \text{ or } B(t)}_{\text{height}} \times \underbrace{(\text{current time} - \text{time of last event})}_{\text{width}}$$

How We Actually Compute the Areas

This is why we track “time of last event”!

The width of each rectangle is always: `clock - time_of_last_event`.

And the height is the *old* (previous) value of the state variable — the value it had *before* this event changes it.

Order Matters!

You must update the area accumulators **before** updating the state variables.

Then update “time of last event” to the current clock value.

Getting this order wrong is one of the most common bugs in simulation!

Let's Simulate by Hand!

Let's get our hands dirty.

We'll trace through every single event, step by step.

You will see exactly what changes and *why*.

Given Data

Interarrival times:

$A_1=0.4, A_2=1.2, A_3=0.5, A_4=1.7, A_5=0.2, A_6=1.6, A_7=0.2, A_8=1.4, A_9=1.9$

Service times:

$S_1=2.0, S_2=0.7, S_3=0.2, S_4=1.1, S_5=3.7, S_6=0.6$

Arrival times (cumulative sums of A_i):

Customer	1	2	3	4	5	6	7	8
A_i	0.4	1.2	0.5	1.7	0.2	1.6	0.2	1.4
Arrival time	0.4	1.6	2.1	3.8	4.0	5.6	5.8	7.2

Goal: Simulate until $n = 6$ customers have completed their delays.

Event e_0 : Initialization ($t = 0$)

Set up the system at time 0:

- Clock = 0, Server status = 0 (idle)
- Number in queue = 0, Arrival list = $\{\}$
- Time of last event = 0

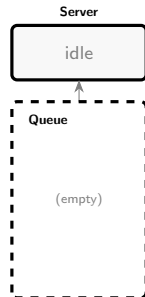
Event list: Next arrival = $0 + A_1 = 0.4$; Next departure = ∞

Counters: # delayed = 0, Total delay = 0,

Area $Q(t) = 0$, Area $B(t) = 0$

Departure = ∞ means “not scheduled.”

Forces next event to be the arrival at $t=0.4$.



Event e_1 : Arrival of Customer 1 ($t = 0.4$)

Area updates (interval = $0.4 - 0 = 0.4$):

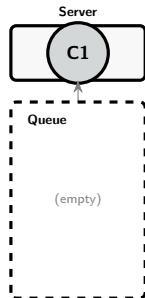
- Area $Q(t)$: $+\underbrace{0}_{\text{old}} \times 0.4 = 0$
- Area $B(t)$: $+\underbrace{0}_{\text{old}} \times 0.4 = 0$

Time of last event: $0 \rightarrow 0.4$.

Schedule:

- Next arrival: $0.4 + 1.2 = \mathbf{1.6}$
- Next departure: $0.4 + 2.0 = \mathbf{2.4}$

$1.6 < 2.4 \Rightarrow$ next event: arrival at $t = 1.6$.



Event e_2 : Arrival of Customer 2 ($t = 1.6$)

Areas ($\Delta t = 1.6 - 0.4 = 1.2$):

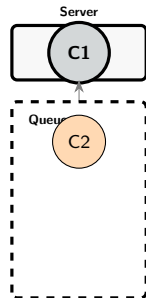
$Q(t)$ was 0: area $+0 \times 1.2 = 0$

$B(t)$ was 1: area $+1 \times 1.2 = 1.2$

Schedule: Next arrival: $1.6 + 0.5 = 2.1$

Departure unchanged at 2.4.

Event list: Arr 2.1, Dep 2.4 $\Rightarrow$ next: arr at 2.1.



Event e_3 : Arrival of Customer 3 ($t = 2.1$)

Clock: $1.6 \rightarrow 2.1$. Customer 3 arrives. Server still busy.

Customer 3 joins queue behind Customer 2.

- # in queue: $1 \rightarrow 2$. Arrival list: $\{1.6, 2.1\}$

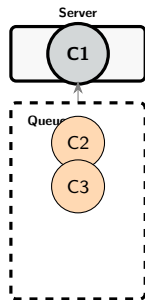
Areas ($\Delta t = 0.5$):

Q : $+1 \times 0.5 = 0.5$ (total: 0.5)

B : $+1 \times 0.5 = 0.5$ (total: 1.7)

Event list: Arr 3.8, Dep 2.4

$\Rightarrow$ next event: **departure at 2.4!**



First Departure!

For the first time, next event is a departure.

Event e_4 : Departure of Customer 1 ($t = 2.4$)

Updates:

- Queue: $2 \rightarrow 1$. Arrival list: $\{2.1\}$
- $S_2 = 0.7 \Rightarrow \text{dep: } 2.4 + 0.7 = 3.1$

Areas ($\Delta t = 0.3$):

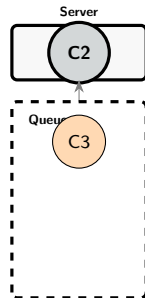
Q: $+2 \times 0.3 = 0.6$ (total: 1.1)

B: $+1 \times 0.3 = 0.3$ (total: 2.0)

Critical Ordering

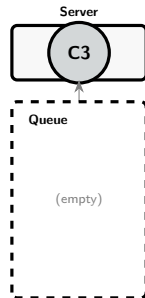
Area uses **old** $Q(t)=2$, then we decrease to 1.

Compute D_2 before advancing the arrival list.



Event e_5 : Departure of Customer 2 ($t = 3.1$)

- Clock: $2.4 \rightarrow 3.1$. C2 departs.
- Queue has C3 $\Rightarrow$ C3 enters service.
- $D_3 = 3.1 - 2.1 = 1.0$. Total delay: 1.8. # delayed: 3.
- $S_3 = 0.2 \Rightarrow$ dep: $3.1 + 0.2 = 3.3$
- Areas ($\Delta t = 0.7$): Q: $+1 \times 0.7 = 0.7$ (total: 1.8);
B: $+1 \times 0.7$ (total: 2.7)



Event e_6 : Departure of C3 ($t = 3.3$) — Server Goes Idle!

Why ∞ ?

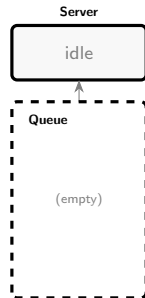
If we left a finite departure time, the timing routine would try to “depart” a nonexistent customer!

Setting $\text{dep} = \infty$ forces the next event to be an arrival.

Areas ($\Delta t = 0.2$): Q : $+0$; B : $+1 \times 0.2 = 0.2$

Totals: $Q = 1.8$, $B = 2.9$.

Event list: Arr 3.8, Dep $\infty \Rightarrow$ next: arr 3.8.



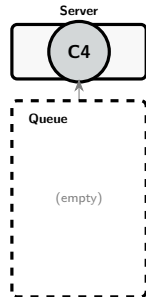
Event e_7 : Arrival of Customer 4 ($t = 3.8$)

C4 arrives to find server **idle**! (Just like C1.)

- $D_4 = 0$. # delayed: 4. Total delay: 1.8.
- Server: $0 \rightarrow 1$. Dep: $3.8 + 1.1 = 4.9$.
- Areas ($\Delta t = 0.5$): Q : +0; B : +0.
(Server was *idle* — $B(t) = 0$!)

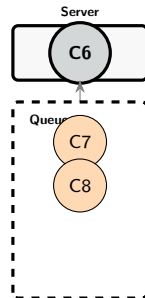
Totals: $Q = 1.8$, $B = 2.9$.

Event list: Arr 4.0, Dep 4.9.



Events e_8 through e_{13} : The Rest

e	t	Type	Q	Delay	#Del
e_8	4.0	Arr C5	$0 \rightarrow 1$	—	4
e_9	4.9	Dep C4	$1 \rightarrow 0$	$D_5=0.9$	5
e_{10}	5.6	Arr C6	$0 \rightarrow 1$	—	5
e_{11}	5.8	Arr C7	$1 \rightarrow 2$	—	5
e_{12}	7.2	Arr C8	$2 \rightarrow 3$	—	5
e_{13}	8.6	Dep C5	$3 \rightarrow 2$	$D_6=3.0$	6



Simulation Stops!

Number delayed reaches $n = 6$. $T(6) = 8.6$.

Main program invokes the **report generator**.

Computing the Final Results

1. Average delay in queue:

$$\begin{aligned}\hat{d}(6) &= \frac{D_1 + D_2 + D_3 + D_4 + D_5 + D_6}{6} \\ &= \frac{0 + 0.8 + 1.0 + 0 + 0.9 + 3.0}{6} = \frac{5.7}{6} = \mathbf{0.95}\end{aligned}$$

On average, a customer waited 0.95 time units before being served.

Computing the Final Results

2. Time-average number in queue:

Total area under $Q(t) = 9.9$ (we accumulated this event by event).

$$\hat{q}(6) = \frac{9.9}{8.6} = \mathbf{1.15}$$

On average, about 1.15 customers were standing in the queue.

Computing the Final Results

3. Server utilization:

Total area under $B(t) = 7.7$ (busy from 0.4–3.3 and 3.8–8.6).

$$\hat{u}(6) = \frac{7.7}{8.6} = \mathbf{0.90}$$

The server was busy 90% of the time.

Computing the Final Results

Caveat

These are **estimates** from *one* simulation run with only $n = 6$ customers. In practice, you'd simulate $n = 1000$ or more, and run *multiple* independent replications.

But the *method* is exactly the same — just more events!

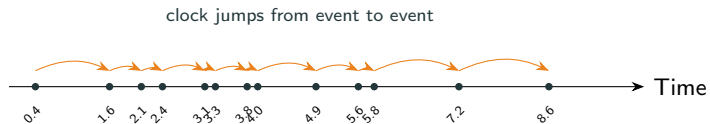
The Clock–Event-List

Critical mechanism in any DES:

At the end of each event, the **timing routine** scans the event list, finds the smallest time, advances the clock to that time, and hands control to the appropriate event routine.

This is how the simulation **progresses through time**. The clock doesn't tick — it *jumps*.

The Clock–Event-List



13 events, 13 jumps. Everything between events is boring — the clock skips it entirely.

Order of Updates Matters!

1. First: Update **area accumulators** (using the *old* values of $Q(t)$ and $B(t)$)
2. Then: Update **state variables** (server status, queue length, arrival list)
3. Then: Update **time of last event** to current clock value
4. Then: Update **event list** (schedule new arrivals/departures)

Common Errors

- Updating $Q(t)$ before computing the area $\Rightarrow$ wrong rectangle height
- Updating time-of-last-event before computing areas $\Rightarrow$ wrong rectangle width
- Forgetting to set departure to ∞ when queue empties $\Rightarrow$ phantom departures

Discrete-Time vs. Continuous-Time Statistics

Our three measures fell into two categories:

Discrete-time statistic: Average delay $\hat{d}(n)$

- Based on a discrete collection of values: $D_1, D_2, \dots, D_n$
- Just a simple arithmetic average
- “Discrete-time” because the index $i = 1, 2, \dots$ is discrete

Discrete-Time vs. Continuous-Time Statistics

Continuous-time statistics: $\hat{q}(n)$ and $\hat{u}(n)$

- Based on functions of continuous time: $Q(t)$ and $B(t)$
- Computed as *area under the curve* divided by total time
- These are *time-weighted* averages

Discrete-Time vs. Continuous-Time Statistics

Why the Distinction Matters

You can't just average the queue lengths observed at event times — that would give a biased result. You need to weight each queue length by *how long* it lasted.

A queue of length 5 that lasts for 10 minutes matters more than a queue of length 5 that lasts for 0.1 seconds.

Before We Walk Further

1. We **initialized** the system (empty and idle)
2. We repeatedly found the **next event** and advanced the clock
3. At each event, we updated **state**, **counters**, and **areas** — in the right order
4. We scheduled **new future events** as needed
5. We stopped when a **condition** was met (6 delays observed)
6. We computed **performance measures** from accumulated statistics

Congratulations!

You just performed a discrete-event simulation by hand. Every computer simulation does *exactly* this — just millions of times faster with millions more events.

(And the computer doesn't complain about the tedium.)

Steps in a Simulation Study

The Big Picture

Simulation is **not** just programming.

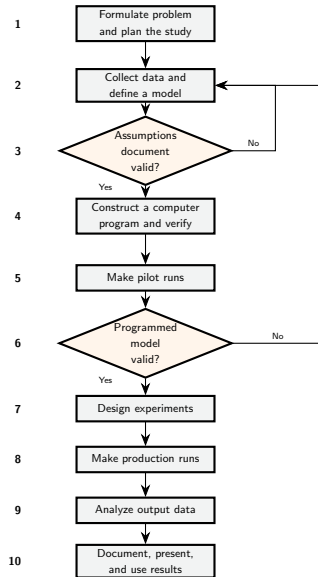
Model programming is just *part* of the overall effort.

Attention must be paid to modeling system randomness, validation, statistical analysis, and project management.

A simulation study is **not a simple sequential process**.

As one proceeds, it may be necessary to go back to a previous step.

Steps in a Simulation Study



Step 1: Formulate Problem and Plan the Study

(a) The problem of interest is stated by the **manager**.

- The problem may not be stated correctly or in quantitative terms
- An *iterative process* is often necessary to pin it down

(b) One or more **kickoff meetings** are conducted, with the project manager, simulation analysts, and **subject-matter experts (SMEs)**.

Step 1: Formulate Problem and Plan the Study

The following are discussed at the kickoff:

- Overall **objectives** of the study
- Specific **questions** to be answered (these determine the level of model detail!)
- **Performance measures** for evaluating different system configurations
- **Scope** of the model
- System **configurations** to be modeled (determines generality of the program)
- **Time frame** for the study and required resources

Step 1: Formulate Problem and Plan the Study

(c) Select the **software** for the model.

- Simulation software (Arena, Flexsim, ProModel) — faster development, lower project cost
- General-purpose language (C, C++, Java, Python) — more control, possibly faster execution

Key Point

The specific questions to be answered by the study *determine* the required level of model detail. Don't decide the detail first.

Step 2: Collect Data and Define a Model

(a) Collect information on the system **structure** and **operating procedures**.

- No single person or document is sufficient!
- Some people may have inaccurate information — make sure true SMEs are identified
- Operating procedures may not be formalized

(b) Collect data to specify model **parameters** and **input probability distributions**.

Step 2: Collect Data and Define a Model

- (c) Delineate all information and data in a written **assumptions document**.
- (d) Collect data on the performance of the **existing system** (for validation in step 6).
- (e) Choosing the level of model detail is **an art**, and depends on:
 - Project objectives and performance measures
 - Data availability and credibility concerns
 - Computer constraints
 - Opinions of SMEs
 - Time and money constraints

Step 2: Collect Data and Define a Model

- (f) There should **not** be a one-to-one correspondence between each element of the model and each element of the real system.
- (g) Start with a **simple model** and embellish as needed.
 - Modeling *every* aspect of the system will seldom be required to make effective decisions
 - Over-modeling may result in excessive execution time, missed deadlines, or obscuring important factors
- (h) Interact with the manager and key personnel on a **regular basis**.

Step 3: Is the Assumptions Document Valid?

- Perform a **structured walk-through** of the assumptions document before an audience of managers, analysts, and SMEs.
- This serves to:
 - Help ensure the model's assumptions are **correct and complete**
 - Promote **interaction** among project members
 - Promote **ownership** of the model
- This walk-through takes place **before programming begins**, to avoid significant reprogramming later.

If "No"

If the assumptions document is *not* valid, go back to step 2. Collect more data, refine the model, revise the document.

Do *not* proceed to programming with invalid assumptions.

Step 4: Construct a Computer Program and Verify

(a) Program the model in a **programming language** (C, C++, Java) or in **simulation software** (Arena, ExtendSim, Flexsim, ProModel).

Trade-offs:

	Programming language	Simulation software
Pros	Known, more control, low cost, faster execution	Less programming, lower project cost
Cons	More coding effort	Less flexibility

Step 4: Construct a Computer Program and Verify

(b) **Verify** (debug) the simulation computer program.

Verification means: *Does the program do what we intended?*

Techniques:

- Code review and structured walk-through
- Trace: print detailed event-by-event output and check by hand
- Check output for reasonableness
- Run under simplifying assumptions where the answer is known

Remember

Verification $\neq$ validation.

Verification: program matches the model. Validation: model matches reality.

Step 5: Make Pilot Runs

- Pilot runs are made for **validation purposes** in step 6.
- These are **preliminary** runs — not the final production runs.
- Use the output to check whether the model behaves as expected.
- Compare against historical data if available.

Think of it as a test drive

You don't take a new car on a 1000-km road trip without driving around the block first. Pilot runs are driving around the block.

Step 6: Is the Programmed Model Valid?

- (a) If there is an **existing system**, compare model and system performance measures for the existing configuration.
- (b) Regardless of whether there is an existing system, the simulation analysts and SMEs should **review the model results** for correctness.

Step 6: Is the Programmed Model Valid?

(c) Use **sensitivity analyses** to determine what model factors have a significant impact on performance measures and, thus, must be modeled carefully.

If “No”

If the programmed model is *not* valid, go back to step 2.

This may seem drastic — going all the way back to step 2! But it makes sense: invalid output often means the conceptual model or the data is wrong, not just the code.

Step 7: Design Experiments

Specify the following for each system configuration of interest:

- **Length of each simulation run** — How much simulated time is needed for accurate estimates?
- **Length of the warmup period**, if appropriate — Many systems start “empty and idle,” which is realistic (Opening time of Bank may contain no clients). We may need to discard initial transient data.
- **Number of independent simulation runs** using different random numbers — Multiple runs allow us to construct confidence intervals and quantify the precision of our estimates.

Steps 8 and 9: Production Runs and Output Analysis

Step 8: Make production runs.

These are the “real” runs, designed according to step 7. The output from these runs is what we use for decision-making.

Unlike pilot runs, production runs use the **full experimental design**: proper run lengths, warmup periods, and multiple replications.

Steps 8 and 9: Production Runs and Output Analysis

Step 9: Analyze output data.

Two major objectives:

1. Determine the **absolute performance** of certain system configurations
 - “What is the expected waiting time for this design?”
2. Compare **alternative configurations** in a relative sense
 - “Is design A better than design B?”
 - “Which of these 5 designs is best?”

Steps 8 and 9: Production Runs and Output Analysis

Statistical Rigor

Simulation outputs are **random** (because inputs are random). You cannot make decisions based on a single number from a single run.

You need:

- **Confidence intervals** — “We are 95% confident the average wait is between 3.2 and 4.1 minutes.”
- **Statistical tests** — “The difference between design A and B is statistically significant.”

Treating simulation output as if it were deterministic is a common and serious error.

Step 10: Document, Present, and Use Results

(a) Document:

- The **assumptions** (from step 2)
- The **computer program** (so others can use and modify it)
- The study's **results** (for current and future projects)

This documentation is critical even if you think the project is “done.” Future analysts will thank you. (Or curse you, if you don't.)

Step 10: Document, Present, and Use Results

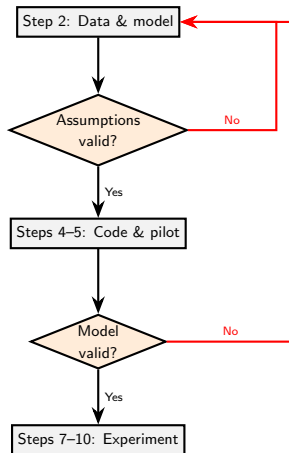
(b) Present the study's results:

- Use **animation** to communicate the model to managers and others not familiar with model details
- Discuss the **model building and validation process** to promote credibility

(c) Results are used in the decision-making process if they are both **valid** and **credible**.

Validity is a technical property. Credibility is a social one — decision-makers must *trust* the results enough to act on them.

The Two Validation Checkpoints



Both “No” Arrows Go Back to Step 2

Not to the immediately previous step — all the way back to data collection and model definition. This is by design: if validation fails, the problem is usually in the conceptual model or the data, not just the code.

Verification and Validation

Why V&V Matters

The most beautiful simulation model in the world is **useless** if it doesn't accurately represent reality.

Simulations, can incorporate *any level of detail*—making them appear realistic on the surface.

This apparent realism can **fool** you.

A wrong model, convincingly animated, gives you **false confidence**.

(A GPS that confidently drives you off a cliff.)

The Goals of V&V

The goal of the validation process is twofold:

1. **Accuracy:** Produce a model that represents true system behavior closely enough to be used as a *substitute* for the actual system—for experimenting, analyzing, and predicting.
2. **Credibility:** Increase confidence in the model to an acceptable level, so that managers and decision makers will actually *use* the results.

Key Insight

Validation is **not** an isolated set of procedures that follows model development. It is an **integral part** of model development, conducted throughout the process.

The Model-Building Process

Building a simulation model involves three main steps, revisited **iteratively**:

1. **Observe the real system:** Collect data, talk to operators, technicians, engineers, supervisors, and managers
2. **Construct a conceptual model:** A collection of assumptions
3. **Implement an operational model:** Translate the conceptual model into simulation code

As model development proceeds, new questions arise, and you return to earlier steps.

The Model-Building Process

The Conceptual Model is a collection of:

1. **Component assumptions:** What are the entities, resources, and their properties?
2. **Structural assumptions:** How do components interact? What simplifications or abstractions are made?
3. **Data assumptions:** Hypotheses about the values of model input parameters and the distributions governing random variables

The **Operational Model** is the computer program that implements these assumptions using simulation software.

Three Key Concepts

Verification

“Did we build the model right?”

Comparing the **conceptual model** to the **computer representation**.

Is the model implemented correctly? Are input parameters and logical structure represented correctly?

Validation

“Did we build the right model?”

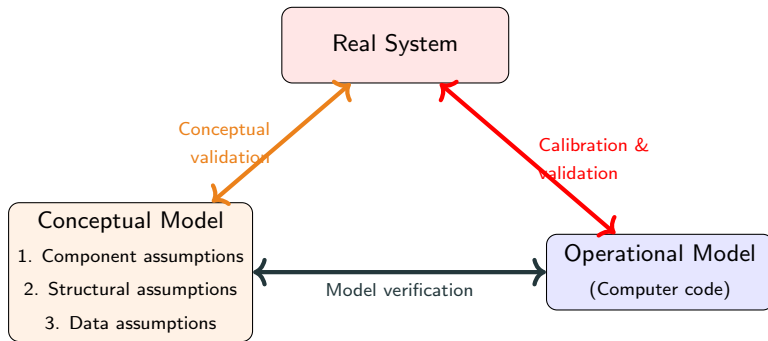
Confirming that the model is an **accurate representation** of the real system.

Does the model's behavior match reality?

Calibration

The iterative engine that drives validation. **Calibration** is the process of comparing the model to the real system, identifying discrepancies, *revising* the model, comparing again, and repeating until accuracy is judged acceptable. Validation is usually *achieved through* calibration.

The V&V Framework



This is an **iterative** process. The model builder goes around this triangle many times, continually comparing the real system to both the conceptual and operational models, modifying each to improve accuracy.

Verification Techniques

The purpose of verification is to assure that the conceptual model is reflected **accurately** in the operational model.

The conceptual model often involves abstraction or simplification.

Verification asks: Are these abstractions and simplifications *correctly represented* in the code?

The following are “common-sense” suggestions—essentially the same practices any good software engineer would follow.

Verification Techniques

- 1. Independent code review:** Have someone other than the developer check the operational model, preferably an expert in the simulation software.
- 2. Flow diagrams:** Include each logically possible action for each event type. Follow the model logic for every action of every event.
- 3. Examine output for reasonableness:** Run the model under a variety of input settings. Display a wide variety of output statistics and examine *all* of them closely.

Verification Techniques

- 4. **Check input parameters:** Have the model print input parameters at the *end* of the simulation to ensure they haven't been changed inadvertently during execution.
- 5. **Self-documentation:** Give a precise definition of every variable, a description of every submodel, procedure, or major code section. Makes it simpler for others (or yourself, later) to verify logic and completeness.
- 6. **Animation:** If the model is animated, watch it! Do things look right?

Verification Techniques

7. Interactive Run Controller (IRC) / Debugger:

An essential component of successful model building:

- Advance the simulation to a desired time or condition, then display model information
- Focus on a particular entity, code line, or procedure—pause whenever that entity becomes active
- Observe values of variables, attributes, queues, resources, counters at any pause point
- Temporarily suspend the simulation to reassign values or redirect entities

Verification Techniques

8. Graphical interfaces: A graphical representation of the model is essentially a form of self-documentation, simplifying the task of understanding the model.

Recommendation

Of all these techniques, the first two (code review and flow diagrams) and documentation should **always** be carried out. Close examination of output for reasonableness is **especially** valuable and informative.

Checking Output Reasonableness

This is the easiest, most valuable, and most often **overlooked** verification technique.

Before running the model, forecast a reasonable range for selected output statistics.

This reduces the temptation to *rationalize* a discrepancy and fail to investigate.

Example: In a complex queueing network, even if you're mainly interested in response time, also collect utilizations, average queue lengths, and waiting times.

Verification by Trace

A **trace** provides detailed, timestamped simulation output whenever an event occurs:

- Values of selected system states
- Entity attributes
- Model variables

The purpose: verify correctness by making detailed **paper-and-pencil calculations** and comparing them against the trace output.

Verification by Trace

Practical considerations:

- A trace over a large time span produces an *enormous* amount of output
- Restrict traces to a **very short period** of time
- Ensure each event type occurs at least once
- For rare events, use *artificial data* to force them to occur—this is legitimate since the goal is verification

Many simulation tools support **selective traces**: triggered at specific locations, specific entities, specific conditions (e.g., “turn on trace when queue ≥ 5 ”).

Example: Catching a Bug with a Trace

Consider a single-server queue. A run over 16 time units gave:

$$\hat{L}_Q = 0.4375 \text{ customers}$$

This seems perfectly reasonable! But the analyst decided to trace. . .

Example: Catching a Bug with a Trace

Trace output:

CLOCK	EVTYP	NCUST	STATUS
0	Start	0	0 (Idle)
red!15 3	Arrival	1	0 (Idle) ← Bug!
5	Depart	0	0 (Idle)
11	Arrival	1	0 (Idle)
12	Arrival	2	1 (Busy)
16	Depart	1	1 (Busy)

At $\text{CLOCK} = 3$: there is 1 customer but the server is **idle**?

That can't be right! The server should have started serving immediately.

Example: Catching a Bug with a Trace

The key lesson:

- The summary statistic $\hat{L}_Q = 0.4375$ looked perfectly reasonable
- In fact, $\hat{L}_Q$ was *computed correctly from the (wrong) data*:

$$\hat{L}_Q = \frac{(1 - 0) \cdot 2 + (0 - 0) \cdot 6 + (1 - 0) \cdot 1 + (2 - 1) \cdot 4}{16} = \frac{7}{16} = 0.4375$$

Moral

The output measure had a reasonable value and was computed correctly, but its value was **wrong** because STATUS was not assuming correct values. The trace caught an error that summary statistics missed entirely.

Calibration and Validation of Models

Calibration and validation, although conceptually distinct, are usually conducted **simultaneously**.

Validation: The overall process of comparing the model and its behavior to the real system and its behavior.

Calibration: The **iterative** process of:

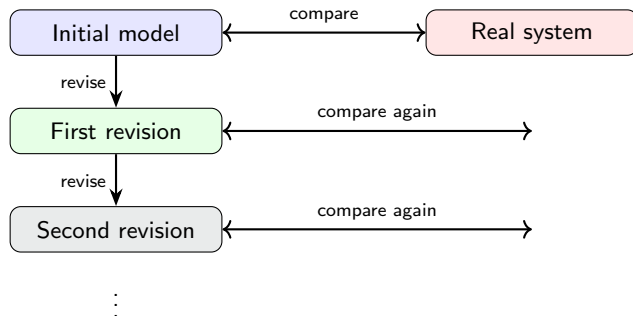
1. Comparing the model to the real system
2. Making adjustments (or even major changes)
3. Comparing the revised model to reality
4. Making additional adjustments
5. Repeating until accuracy is judged sufficient

Calibration and Validation of Models

The comparison is carried out by a variety of tests:

- **Subjective tests:** Knowledgeable people make judgments about the model and its output
- **Objective tests:** Require data on the system's behavior and corresponding model data, compared using statistical tests

The Calibration Loop



Validation is NOT Binary

No model is ever 100% valid. Each revision costs time and money.

The question: Is it **accurate enough** for the decisions we need to make?

Calibration and Validation Data

A possible criticism: “You’ve just *fitted* the model to one specific dataset!”

To alleviate this:

1. Collect system data
2. Split into two sets:
 - **Calibration set:** Used to build and tune the model
 - **Validation set:** Reserved for final testing
3. Calibrate the model using the first set
4. Test the calibrated model against the second set

Calibration and Validation Data

If unacceptable discrepancies are found in the “final” validation:

- Return to the calibration phase
- Modify the model until it becomes acceptable

This approach shows the model works on data it *has never seen before*, not just the data it was tuned on.

The Naylor–Finger [1967] Three-Step Approach

A widely followed framework for validation:

1. **Face validity**: Build a model that *appears* reasonable to knowledgeable people
2. **Validate model assumptions**: Are the structural and data assumptions correct?
3. **Validate input-output transformations**: Does the model produce outputs consistent with the real system's outputs?

Let's examine each step in detail.

Step 1: Face Validity

Show your model to people who **know the real system**.

Ask them: “Does this seem right?”

Potential users should be involved from **conceptualization to implementation**:

- They ensure reasonable assumptions about system structure
- They supply reliable data
- They catch deficiencies you'd never notice
- They evaluate output for reasonableness

Step 1: Face Validity

Another advantage of user involvement:

- Increases the model's **perceived validity** (credibility)
- Without credibility, managers will *not* trust the results as a basis for decision making
- Users develop *ownership* of the model

This is subjective, but extremely valuable.

Step 1: Face Validity

Sensitivity analysis for face validity:

Ask the model user: “Does the model behave as you’d expect when an input variable is changed?”

- Increase the arrival rate $\Rightarrow$ do utilizations, queue lengths, and delays increase?
- Add a server $\Rightarrow$ do delays decrease?

If the model behaves counter-intuitively, something may be wrong.

If real system data are available for at least two settings of input parameters, objective statistical sensitivity tests can be conducted.

Step 2: Validating Model Assumptions

Two classes of assumptions to validate:

Structural assumptions: How is the system organized?

- Is there one queue or many? Do customers switch lines?
- Is the number of servers fixed or variable?
- What is the queue discipline?

Verify these by *actual observation* during appropriate time periods and by *discussions* with managers and operators.

Step 2: Validating Model Assumptions

Data assumptions: What distributions govern the random variables?

Three-step statistical analysis:

1. **Identify** an appropriate probability distribution
2. **Estimate** the parameters of the hypothesized distribution
3. **Validate** by a goodness-of-fit test (chi-square, Kolmogorov–Smirnov) and graphical methods

Step 2: Validating Model Assumptions

Data Reliability

- Cross-check with multiple sources. Operators may have inaccurate information.
- When combining data sets collected at different times, test for **homogeneity** first (do the two sets come from the same population?).
- Test for **correlation** in the data. Only proceed with standard analysis once you are assured of having a random sample.

Step 3: Validating Input-Output Transformations

Test of the model as a whole.

The model is viewed as an input-output transformation:

$$f(\mathbf{X}, \mathbf{D}) = \mathbf{Y}$$

- **X**: uncontrollable input variables (e.g., arrival times, service times)
- **D**: decision variables or controllable parameters (e.g., number of servers)
- **Y**: output or response variables (e.g., utilization, delay)
- f : the transformation

Step 3: Validating Input-Output Transformations

The question: When the model receives the same (or statistically equivalent) inputs as the real system, does it produce outputs consistent with the real system's outputs?

Requirement: Some version of the system must *exist* so that system data can be collected.

If the system is still in planning stages and no operating data can be collected, complete input-output validation is *not* possible.

Validating Models of New/Modified Systems

Often the model will be used to compare *alternative* system designs or investigate behavior under *new* input conditions.

If the existing system model has been validated, can we **transfer confidence** to the model of a proposed (nonexistent) system?

Generally yes—if the new model is a relatively **minor modification** of the old one in terms of changes to the operational model.

Validating Models of New/Modified Systems

Changes from minor to major:

1. **Minor:** Change a single parameter (machine speed, arrival rate, number of servers)
2. **Minor:** Change the form of a distribution (service time distribution)
3. **Major:** Change logical structure of a subsystem (queue discipline, scheduling rule)
4. **Major:** Completely different system design (e.g., computerized inventory replacing manual)

For minor changes (1–2): verify carefully, accept output with considerable confidence.

For major changes (3–4): partial validation may be possible if similar subsystems exist elsewhere.

There is no way to completely validate the input-output transformations of a model of a nonexistent system.

Thank you!